

BankAI RAG Platform

A Secure, Evaluated Retrieval-Augmented Generation Architecture for Banking Intelligence

Technical Research Paper

Author: Maksud Bhaldar

Repository: github.com/macbhaldar/bankai-rag-platform

Version: Research/Engineering Documentation 2026

Abstract

BankAI RAG Platform is a modular research and engineering implementation for banking-oriented information retrieval, question answering, structured analytics, and access-controlled knowledge delivery. The platform combines hybrid retrieval- dense embeddings and BM25 lexical search- with Reciprocal Rank Fusion (RRF), optional reranking, metadata filtering, document-family deduplication, and principal-aware access control. A separate structured-data path supports controlled natural-language-to-SQL analytics with read-only execution and row limits. The generation layer supports OpenAI-compatible endpoints, Ollama, and an extractive offline provider, allowing the system to remain testable without mandatory external model access. Security controls include permission-aware retrieval, document grants, department/domain restrictions, PII masking, audit events, configurable API-key authentication, and SQL safety controls. Evaluation infrastructure covers retrieval and question-answering benchmarks, including Recall@K, MRR, nDCG, document/section hit rates, family-level retrieval, and hard-negative robustness. This paper describes the architecture, methodology, implementation, evaluation design, security model, limitations, and research roadmap of the project. Because the repository is primarily a reference and prototype platform, the paper distinguishes implemented functionality from production controls that remain future work.

Keywords: Retrieval-Augmented Generation; RAG; banking AI; hybrid retrieval; BM25; dense retrieval; Reciprocal Rank Fusion; access control; PII protection; natural-language-to-SQL; LLM evaluation; auditability

1. Introduction

Financial institutions manage large volumes of policies, procedures, compliance material, product documentation, customer and transaction data, and operational records. Conventional keyword search can be insufficient when users formulate questions using terminology that differs from the source documents. Large language models can improve natural-language interaction, but unconstrained generation creates risks including hallucination, leakage of restricted information, unsupported claims, and unsafe database access.

Retrieval-Augmented Generation addresses part of this problem by separating evidence retrieval from language generation. Instead of relying exclusively on parametric model memory, the system retrieves relevant external evidence and supplies it as context to the generator. For banking applications, however, retrieval quality alone is insufficient. Authorization must be applied before restricted information reaches the generation layer, structured

analytics must be isolated from document retrieval, and system behavior must be measurable through repeatable evaluation.

BankAI RAG Platform was developed around these requirements. Its central research question is: how can a modular RAG architecture combine retrieval effectiveness, security controls, structured analytics, provider independence, offline operation, and evaluation in one maintainable banking-oriented system?

2. Objectives and Research Questions

- Design a modular banking-oriented RAG architecture that separates ingestion, retrieval, generation, security, evaluation, API, and UI responsibilities.
- Improve retrieval robustness through complementary lexical and semantic retrieval rather than depending on a single search method.
- Prevent unauthorized documents from entering the RAG context by applying principal-aware access control before generation.
- Provide a controlled structured-data workflow for analytical questions without allowing unrestricted database writes.
- Support multiple LLM backends and an extractive fallback to reduce vendor and network dependency.
- Make retrieval and QA behavior measurable through benchmark datasets and standard ranking metrics.
- Provide auditability and PII controls suitable for research into trustworthy banking information systems.

The principal research questions are: (RQ1) Does hybrid retrieval provide a stronger retrieval basis than a single retrieval strategy? (RQ2) Can authorization be integrated before context construction without changing the generation interface? (RQ3) Can structured analytics be isolated into a safer execution path? (RQ4) How can an evaluation harness expose retrieval regressions and hard-negative failures? (RQ5) Which controls remain necessary before a prototype can be considered suitable for regulated production?

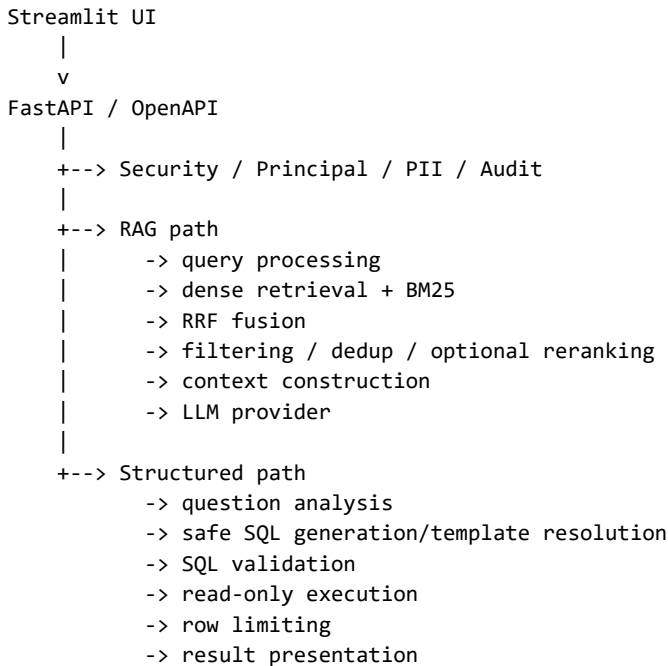
3. System Scope

The repository implements a Python-based application with a FastAPI service layer and Streamlit interface. It supports document ingestion for formats including Markdown, TXT, PDF, DOCX, CSV, and JSON; configurable chunking; embeddings; vector retrieval; BM25 retrieval; hybrid ranking; optional reranking; metadata filtering; RAG generation; structured analytics; evaluation; and security/audit components.

Layer	Primary responsibility	Representative implementation
Ingestion	Load, normalize, chunk and index documents	app/pipelines
Retrieval	Semantic and lexical candidate generation	app/retrieval
Generation	Provider abstraction and QA	app/generation
Security	ACL, PII, authentication, audit	app/security and app/core
Structured analytics	Controlled NL-to-SQL workflow	app/structured
API	HTTP endpoints and request validation	app/api
UI	Interactive research/operator interface	app/ui
Evaluation	Benchmark execution and metrics	app/evaluation

4. Architecture

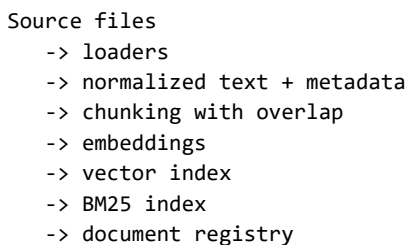
The architecture follows a layered design. A Streamlit client communicates with FastAPI. The API routes requests into either the unstructured RAG path or the structured analytics path. Cross-cutting security and audit mechanisms operate around these paths.



This separation is important because document retrieval and database analytics have different threat models. A document question primarily requires evidence selection and access filtering. A structured query additionally requires control over generated executable statements, database permissions, resource usage, and result exposure.

5. Data and Ingestion Pipeline

The ingestion subsystem transforms heterogeneous source material into searchable units. The conceptual pipeline is loader → normalization → metadata extraction → chunking → embedding/indexing. Metadata can support document family, category, source, and access-related filtering.



Chunking is a key design decision. Very large chunks can dilute semantic relevance and increase context cost, while very small chunks can remove necessary context. Overlap helps preserve information that crosses chunk boundaries. The platform therefore exposes chunking configuration rather than hard-coding a single strategy.

6. Hybrid Retrieval Methodology

The retrieval subsystem combines dense and lexical retrieval. Dense retrieval represents queries and documents in an embedding space and is useful when a user's wording differs from the wording in the source. BM25 is a lexical

ranking method that rewards term frequency and inverse document frequency and is useful for exact names, policy identifiers, domain terms, and uncommon phrases.

The platform combines rankings with Reciprocal Rank Fusion. If a document has rank r in a retrieval list, a simplified RRF contribution is:

$$RRF(d) = \sum_i 1 / (k + rank_i(d))$$

where i indexes retrieval systems and k is a stabilizing constant. The exact implementation parameters should be treated as configuration rather than assumed universal values. Fusion allows a document to obtain evidence from multiple retrieval mechanisms.

After candidate generation, the platform can apply metadata filtering, document-family deduplication, principal-aware access control, and optional reranking. This sequence is especially important for security: access filtering must occur before restricted content is inserted into the generation context.

7. Retrieval-Augmented Generation

RAG separates evidence acquisition from natural-language synthesis. A typical request follows: question → retrieval → evidence selection → prompt construction → generation → response with source context. The generator is therefore not expected to answer solely from its learned parameters.

The platform's provider abstraction supports OpenAI-compatible services, Ollama, and an extractive provider. The extractive implementation is particularly useful for deterministic testing because it can return relevant retrieved sentences without requiring an external generative model. This provides a useful control condition when testing retrieval or API behavior.

A key engineering principle is retrieval before generation. If retrieval returns poor evidence, changing the language model may not solve the underlying information-access problem. Evaluation therefore needs to isolate retrieval performance from generation quality.

8. Security Architecture

Banking information systems require authorization boundaries that are stronger than a conventional search interface. BankAI RAG Platform treats security as part of the retrieval architecture.

The central flow is:

```
User / principal
-> resolve permissions
-> determine allowed document scope
-> retrieve within authorized scope
-> construct context
-> generate answer
```

This design reduces the risk that a user will receive a response containing information from a document that they were not authorized to access. Additional controls include department/domain restrictions, document-level grants, a break-glass access model, PII masking, audit events, configurable API-key authentication, read-only structured SQL, SQL safety validation, and row limits.

The API authentication layer can require X-API-Key for /api/* routes when BANKRAG_API_KEY is configured, while keeping the health endpoint public for operational monitoring. This is appropriate for a development/reference architecture but should not be interpreted as a complete enterprise identity system.

9. Structured Analytics and Natural-Language-to-SQL

Banking users frequently need aggregate or relational answers such as branch exposure, transaction totals, loan distributions, or account counts. These questions are fundamentally different from document lookup. The platform therefore implements a separate structured path.

```
Natural-language question
  -> analysis / template resolution
  -> SQL generation
  -> SQL validation
  -> read-only execution
  -> row limit
  -> human-readable response
```

The security objective is not merely to produce syntactically valid SQL. The system must prevent writes and other unsafe statements, limit resource consumption, and control the amount of data returned. In a production environment, database-level permissions should complement application validation rather than relying on an application validator alone.

10. Evaluation Framework

Evaluation is designed around retrieval quality and QA robustness. Standard ranking metrics include Recall@K, Mean Reciprocal Rank (MRR), and normalized Discounted Cumulative Gain (nDCG).

Recall @ K = relevant documents retrieved $\in$ top K / total relevant documents

MRR = mean(1 / rank of first relevant result)

DCG @ K = $\sum_i (2^{rel_i} - 1) / \log_2(i + 1)$

nDCG @ K = DCG @ K / ideal_{DCG} @ K

The repository additionally evaluates document hit rate, section hit rate, family-level retrieval, hard-negative robustness, and QA benchmark behavior. Hard negatives are important because banking corpora may contain closely related policies, products, procedures, and regulatory material. A system that performs well on easy queries may still fail when documents differ only in a small but decisive clause.

Evaluation reports are designed to be stored as machine-readable JSON so that experiments can be compared programmatically. A mature experimental workflow should report dataset version, retrieval configuration, embedding model, chunking configuration, reranker configuration, K values, and evaluation timestamp alongside scores.

11. Implementation and Software Engineering

The project is organized into focused modules. Retrieval logic is separated from generation; structured analytics is separated from document RAG; security is separated into an explicit package; and API/UI code acts as an integration boundary. This reduces coupling and makes provider substitution possible.

The project also includes unit tests and a GitHub Actions CI workflow targeting multiple Python versions. The development workflow recommended by the project is test → ingest when necessary → evaluate → inspect security/audit behavior → commit → CI.

Engineering property	Implementation rationale
Modularity	Independent retrieval, generation, security, structured and UI modules
Provider independence	OpenAI-compatible, Ollama and extractive providers
Testability	Offline extractive mode and unit tests
Reproducibility	Configuration files and benchmark scripts
Observability	Audit subsystem and evaluation reports
Maintainability	FastAPI boundary and typed application schemas

12. Experimental Methodology

The repository provides the infrastructure for experiments, but this paper does not invent benchmark scores that have not been measured. A rigorous experimental study should compare at least the following configurations:

1. BM25-only retrieval.
2. Dense-only retrieval.
3. Hybrid BM25 + dense retrieval with RRF.
4. Hybrid retrieval with reranking.
5. Hybrid retrieval with metadata and principal-aware filtering.

For each configuration, use the same benchmark queries, relevance judgments, document corpus, chunking policy, and evaluation K values. Report Recall@K, MRR, nDCG, document hit rate, section hit rate, latency, and—where applicable—answer grounding/faithfulness measures.

Security experiments should create at least two principals with intentionally different document permissions. The test should verify that restricted documents never appear in retrieved context or final answers for unauthorized principals. Structured-query experiments should include valid analytical queries and adversarial requests containing INSERT, UPDATE, DELETE, DROP, or other disallowed operations.

13. Threat Model

Threat	Potential impact	Primary mitigation
Unauthorized document retrieval	Confidentiality breach	Principal-aware ACL filtering before context construction
PII exposure	Privacy breach	PII masking and access controls
Prompt injection in documents	Instruction hijacking	Future dedicated prompt-injection detection and policy enforcement
Unsafe SQL generation	Data modification or disclosure	Read-only execution, SQL validation, row limits
API key compromise	Unauthorized API use	Secrets management and stronger identity in production
Hallucination	Incorrect operational decision	Evidence-based RAG and

		grounding evaluation
Retrieval of near-duplicate wrong policy	Incorrect answer	Hybrid retrieval, reranking, family-aware evaluation
Model/provider outage	Service degradation	Provider abstraction and extractive fallback

14. Reliability and Failure Modes

A RAG application can fail even when the LLM is functioning correctly. Retrieval may select an obsolete document, a semantically similar but unauthorized document, or an incomplete chunk. Chunking may separate a definition from its exception. A reranker may improve relevance for one query family while degrading another. The generation model may also synthesize unsupported conclusions from partially relevant evidence.

The architecture therefore treats retrieval evaluation, access control, and auditability as first-class components. Production deployments should add document versioning and freshness policies so that superseded policies cannot silently remain competitive in retrieval.

Structured analytics has a different failure profile: ambiguous user questions, incorrect schema assumptions, unsafe SQL, expensive queries, and excessive result sets. The separate structured path makes it possible to apply controls specific to these risks.

15. Performance Considerations

Performance depends on corpus size, embedding dimensionality, index implementation, candidate counts, reranking cost, LLM latency, and hardware. Hybrid retrieval adds work because more than one candidate-generation mechanism is executed. Reranking can further increase latency but may improve top-K precision.

For a production benchmark, report end-to-end latency as well as component latency: query processing, dense retrieval, BM25 retrieval, fusion, reranking, prompt construction, LLM generation, and structured SQL execution. Memory usage and index-build time should also be measured.

The current platform is primarily a modular reference implementation rather than a distributed high-throughput serving system. Scaling to large enterprise corpora would likely require persistent databases, distributed vector storage, asynchronous ingestion, caching, rate limiting, and observability infrastructure.

16. Discussion

The principal architectural contribution of BankAI RAG Platform is the combination of several concerns that are often implemented independently: hybrid retrieval, provider abstraction, access-aware retrieval, structured analytics, evaluation, and auditability. In a banking setting, these concerns interact. Retrieval quality without authorization can expose sensitive information; authorization without evaluation can silently degrade user utility; generation without structured-data isolation can create unsafe database behavior.

The platform's offline provider is also methodologically useful. It allows the retrieval and API layers to be tested independently of a proprietary or hosted LLM. This is particularly valuable for CI and reproducible engineering experiments.

At the same time, the platform should not be presented as a production-ready banking system. A real deployment would require enterprise identity, encryption, formal governance, secure secret storage, network controls, durable

audit infrastructure, comprehensive threat modeling, penetration testing, model risk management, incident response, disaster recovery, and regulatory review.

17. Limitations

- The repository does not by itself establish production-level compliance with banking regulations.
- Evaluation infrastructure exists, but benchmark results must be generated from controlled experiments rather than inferred from code presence.
- API-key authentication is not a substitute for enterprise SSO/OIDC, authorization services, or mTLS.
- Prompt-injection resistance requires additional dedicated controls and adversarial testing.
- PII masking is an application control and should be combined with data minimization, database permissions, encryption, and retention policies.
- Local vector/index components are not equivalent to a distributed production retrieval service.
- LLM output quality and factuality remain dependent on the selected model and evidence quality.

18. Future Research Directions

- Compare BM25, dense, hybrid, and reranked retrieval under identical benchmark conditions.
- Study retrieval degradation under corpus growth and policy-version changes.
- Develop document-version and temporal retrieval so that historical policy questions retrieve the correct effective version.
- Evaluate fine-grained ABAC policies and policy-composition performance.
- Measure the effect of query rewriting, multi-query retrieval, and query decomposition.
- Develop adversarial prompt-injection and indirect-instruction benchmarks.
- Add grounding and citation-completeness metrics for generated answers.
- Measure latency/cost trade-offs between local and hosted LLM providers.
- Study human-in-the-loop review for high-risk banking questions.
- Integrate OpenTelemetry, centralized audit, monitoring, and reproducible experiment tracking.

19. Reproducibility Guide

A baseline local experiment can be reproduced from the repository using the documented installation, offline configuration, ingestion, evaluation, and test commands.

```
py -3 -m venv .venv
.venv\Scripts\python.exe -m pip install -r requirements.txt
copy .env.example .env
python scripts/run_ingest.py --rebuild
pytest -q
python scripts/run_eval.py --benchmark retrieval
python scripts/run_eval.py --benchmark qa_test
python scripts/run_eval.py --benchmark hard_negatives
```

For a publishable experimental report, preserve the exact repository commit, dataset version, configuration file, Python version, model identifiers, benchmark inputs, and generated evaluation reports. This prevents results from becoming ambiguous after future repository changes.

20. Conclusion

BankAI RAG Platform demonstrates a practical architecture for banking-oriented RAG in which retrieval, generation, structured analytics, security, and evaluation are treated as coordinated engineering components. Its hybrid retrieval strategy combines semantic and lexical evidence; its provider layer supports both external and local generation; its extractive fallback enables offline testing; its structured path isolates database analytics; and its security layer places authorization before context construction.

The most important conclusion is architectural rather than numerical: a trustworthy banking RAG system cannot be evaluated solely by the fluency of its answers. Retrieval relevance, authorization correctness, data protection, structured-query safety, auditability, latency, and reproducibility all require explicit engineering and measurement. The current repository provides a foundation for those experiments while clearly leaving enterprise production controls as future work.

References

Lewis, P. et al. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. *Advances in Neural Information Processing Systems*.

Robertson, S. and Zaragoza, H. (2009). The Probabilistic Relevance Framework: BM25 and Beyond. *Foundations and Trends in Information Retrieval*.

Carbonell, J. and Goldstein, J. (1998). The Use of MMR, Diversity-Based Reranking for Reordering Documents and Producing Summaries. *SIGIR*.

Page, L. et al. (1999). The PageRank Citation Ranking: Bringing Order to the Web. Stanford technical report. (Included as general ranking background; not a dependency of the platform.)

OpenAI. OpenAI API documentation and model-provider concepts. Consult the provider documentation used by a deployment.

FastAPI documentation. API framework used by the project.

Streamlit documentation. Interactive application framework used by the project.

GitHub repository: BankAI RAG Platform — [macbhaldar/bankai-rag-platform](https://github.com/macbhaldar/bankai-rag-platform).

Appendix A — Repository-to-Paper Mapping

Research topic	Repository location
API and integration	app/api
Configuration and core services	app/core, config
Document ingestion	app/pipelines
Embeddings and vector retrieval	app/retrieval/embeddings.py, vector_store.py
BM25 and hybrid retrieval	app/retrieval/bm25.py, hybrid.py
Reranking	app/retrieval/reranker.py
LLM providers	app/generation/llm.py
Question answering	app/generation/qa.py
ACL/security	app/security/acl.py
PII and audit	app/core/pii.py, app/core/audit.py
Structured analytics	app/structured
Evaluation	app/evaluation, scripts/run_eval.py
User interface	app/ui
Automated testing	tests
Continuous integration	.github/workflows/ci.yml

Appendix B — Example Research Experiment Matrix

Experiment	Independent variable	Primary measures
E1	BM25 vs dense vs hybrid	Recall@K, MRR, nDCG
E2	RRF vs reranking	MRR, nDCG, latency
E3	ACL enabled vs bypassed in controlled test	Unauthorized-hit rate
E4	Easy vs hard-negative queries	Recall@K, family hit rate
E5	Extractive vs generative provider	Latency, grounding, answer quality
E6	Chunk sizes / overlap	Retrieval metrics, context size
E7	Structured-query safety	Blocked unsafe-query rate, valid-query success rate